UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ · CÂMPUS CORNÉLIO PROCÓPIO

PROJETO REA · LICENÇA MIT

>_ TRY, CATCH & LEARN

Fundamentos de Programação em C

Da História da Computação e Memória RAM às Matrizes e Structs: Um Livro-Texto Aberto, Visual e Passo a Passo

EQUIPE AUTORA
Lauren Marçulo · Lorena Eduarda · Manuella Vieira · Renan Cáceres

ORIENTADOR RESPONSÁVEL
Prof. Dr. André Luís dos Santos Domingues

INSTITUIÇÃO E PROJETO
UTFPR-CP · Certificadora de Competência Comum

AMBIENTE DE PRÁTICA
OnlineGDB (Compilador Oficial GCC em Nuvem)

Obra didática desenvolvida como Recurso Educacional Aberto (REA) para estudantes ingressantes em cursos de Computação, Engenharias e Tecnologia. Reúne a fundamentação histórica da ciência da computação, explicações detalhadas da arquitetura de memória, testes de mesa comentados linha por linha, diagramas vetoriais e referências bibliográficas clássicas.

Edição Completa Expandida · Novembro de 2026

Cornélio Procópio, Paraná, Brasil

APRESENTAÇÃO INSTITUCIONAL E PEDAGÓGICA

A Equipe e o Propósito do Projeto Try, Catch & Learn

Por que decidimos escrever este livro aberto e como nossa metodologia combate a evasão no primeiro semestre.

Quem entra em um curso de Computação ou Engenharia costuma carregar uma mistura de entusiasmo e receio. Nas primeiras semanas de aula, quando o quadro-negro se enche de chaves, pontos-e-vírgulas, endereços hexadecimais e mensagens em inglês vindas do compilador, muitos estudantes começam a duvidar da própria capacidade. Não é coincidência que as disciplinas introdutórias de Algoritmos e Fundamentos de Programação registrem, há décadas, índices elevados de reprovação e desistência nas universidades brasileiras.

Observando de perto essa realidade nos laboratórios da **Universidade Tecnológica Federal do Paraná (UTFPR — Câmpus Cornélio Procópio)**, percebemos que a dificuldade principal da maioria dos calouros não está na lógica matemática em si, mas na falta de uma ponte entre o pensamento cotidiano e o funcionamento físico do computador. Livros tradicionais muitas vezes foram escritos assumindo que o leitor já conhece jargões técnicos, enquanto tutoriais soltos na internet ensinam a copiar comandos sem explicar o que realmente acontece dentro da memória RAM.

Por que escolhemos o nome "Try, Catch & Learn"?

Em linguagens mais recentes como Java, C# ou Python, existe uma estrutura chamada `try / catch` usada para tratar exceções sem que o programa trave. Na linguagem C pura, essa estrutura não existe no código, porque quem faz esse papel é o próprio estudante.

Nosso nome resume o ciclo de estudo que defendemos em todas as nossas videoaulas, listas de exercícios e neste livro:

- **Try (Tentar):** Perder o medo da tela em branco. Você só aprende a programar escrevendo código, testando hipóteses e alterando variáveis para ver como o computador reage.
- **Catch (Capturar e Investigar):** Parar de enxergar o aviso vermelho do compilador como uma punição. Uma mensagem de erro nada mais é do que uma coordenada exata (linha e coluna) indicando onde a gramática da linguagem precisa de ajuste.
- **Learn (Aprender com Modelo Mental):** Consolidar o conceito entendendo como os bytes foram guardados nas gavetas da memória RAM, para nunca precisar decorar código mecanicamente.

Compromisso com a Educação Aberta (Pilar REA e Licença MIT)

Seguindo as diretrizes da **Declaração REA de Paris (UNESCO, 2012)**, todo o ecossistema do projeto *Try, Catch & Learn* — incluindo nosso portal web responsivo, videoaulas, slides interativos, cadernos de missões comentados e este livro — foi publicado como um **Recurso Educacional Aberto (REA)** sob a licença **MIT**. Isso garante os chamados "5 Rs" da educação aberta: qualquer estudante ou professor tem permissão legal e gratuita para **Reter** (baixar e guardar sua cópia), **Reutilizar** em salas de aula ou grupos de estudo, **Revisar**, **Remixar** e **Redistribuir** este material sem barreiras financeiras ou tecnológicas.

Os Autores do Projeto



Lauren Marçulo

DIDÁTICA, EXERCÍCIOS & PROGRESSÃO

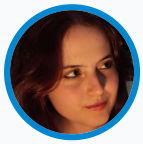
Responsável pela criação das dinâmicas lúdicas, roteirização pedagógica das fases de aprendizagem e elaboração de problemas contextualizados que conectam a sintaxe da linguagem C a situações práticas do dia a dia.



Lorena Eduarda

DESIGN EDITORIAL & IDENTIDADE VISUAL

Responsável pela direção visual do projeto, concepção dos diagramas didáticos, tipografia acessível, organização visual dos slides das Fases 1 a 6 e experiência de leitura tanto no computador quanto no celular.



Manuella Vieira

CURADORIA TEÓRICA & DOCUMENTAÇÃO REA

Responsável pela pesquisa bibliográfica, estruturação conceitual dos capítulos teóricos, padronização da linguagem didática e adequação do material aos princípios de acessibilidade dos Recursos Educacionais Abertos.



Renan Cáceres Anselmo

ARQUITETURA DE CÓDIGO C & PLATAFORMA WEB

Responsável pelo desenvolvimento e validação de todos os algoritmos em C no compilador GCC, criação dos testes de mesa, representação física da memória RAM e implementação do portal web no GitHub Pages.

ORIENTAÇÃO E SUPERVISÃO ACADÊMICA

Prof. Dr. André Luís dos Santos Domingues

Docente Orientador · Certificadora de Competência Comum · UTFPR Câmpus Cornélio Procópio

ROTEIRO DE ESTUDOS

Sumário Geral e Mapa das Fases

Como este livro se conecta às 6 Fases de videoaulas e listas de exercícios do nosso curso.

Este livro foi organizado em **14 capítulos progressivos**. Os três primeiros capítulos constroem a base conceitual e prática: de onde vieram os computadores, como o código escrito em texto se transforma em pulsos elétricos através do compilador GCC e como dominar o ambiente **OnlineGDB** sem instalar nada. Do Capítulo 4 ao Capítulo 12, percorremos todas as **6 Fases** oficiais do curso *Try, Catch & Learn*. Por fim, os Capítulos 13 e 14 trazem nosso guia prático de resolução de problemas e as referências bibliográficas completas para aprofundamento.

Capítulo	Etapa do Curso	Tópicos Detalhados no Capítulo	Ferramenta Mental
Capítulo 1	Base Histórica	**As Raízes da Computação: Dos Teares aos Laboratórios Bell** Tear de Jacquard (1804), Ada Lovelace e Babbage (1843), Máquina de Turing (1936), transistores, UNIX e a criação de C por Dennis Ritchie (1972).	Evolução Binária e Portabilidade
Capítulo 2	Arquitetura	**O que é uma Linguagem de Programação e Como o Compilador Funciona** Transistores e código binário; compilação vs. interpretação; as 4 etapas do GCC (Pré-processador, Compilador, Assembler e Linker) e tipos de erros.	Pipeline de Compilação GCC
Capítulo 3	Laboratório	**O Ambiente de Desenvolvimento: Dominando o OnlineGDB** O que é uma IDE; por que evitamos instalações locais complexas; anatomia do OnlineGDB, console interativo e uso passo a passo do Debugger (Breakpoints).	Depuração Visual em Nuvem
Capítulo 4	Fase 1 · Parte A	**Anatomia Completa de um Programa em C** Diretivas `#include`, função `main()`, chaves de bloco, ponto-e-vírgula, comentários, sequências de escape (`\n`, `\t`) e erros clássicos.	Esqueleto Padrão ANSI C
Capítulo 5	Fase 1 · Parte B	**Memória RAM, Variáveis, Tipos Primitivos e Operadores** As 4 propriedades de uma variável; `int`, `float`, `double` e `char`; lixo de memória; constantes; precedência aritmética, módulo (`%`) e Type Casting.	Modelo das Gavetas na RAM
Capítulo 6	Fase 1 · Parte C	**Entrada e Saída de Dados: `printf`, `scanf` e o Buffer do Teclado** Formatação de casas decimais; o papel físico do operador de endereço `&`; como funciona a fila do teclado (`stdin`) e solução do bug do Enter residual.	Metáfora do Carteiro (`&`)
Capítulo 7	Fase 2	**Estruturas de Decisão: Ensinando o Programa a Escolher** Operadores relacionais e tabelas-verdade lógicas (`&&`, `||`, `!`); desvios `if`, `else if` e `else`; armadilha do `=` vs `==`; seleção múltipla com `switch-case`.	Árvore de Decisão Booleana
Capítulo 8	Fase 3	**Estruturas de Repetição: Laços `while`, `do-while`, `for` e Teste de Mesa** Teste no início vs. teste no final; os 4 estágios do `for`; contadores e acumuladores; rastreamento manual linha a linha (Teste de Mesa) e loops infinitos.	Ciclo de Iteração e Rastreio
Capítulo 9	Fase 4	**Modularização e Funções: Dividir para Conquistar** Por que modularizar; funções com retorno vs. procedimentos `void`; passagem de parâmetros por valor (cópia na pilha de memória) e escopo de variáveis.	Caixas-Pretas Independentes

Capítulo	Etapa do Curso	Tópicos Detalhados no Capítulo	Ferramenta Mental
Capítulo 10	Fase 5 · Parte A	**Vetores (Arrays Unidimensionais) e Manipulação de Strings** Alocação contígua em base zero (`0` a `N-1`); invasão de memória; varredura com `for` ; strings terminadas em `'\0'` , leitura com espaços e `<string.h>` .	Fileira Indexada na RAM
Capítulo 11	Fase 5 · Parte B	**Matrizes (Arrays Bidimensionais): Tabelas, Grades e Jogos** Coordenadas de linha `[i]` e coluna `[j]` ; sincronismo dos laços `for` aninhados; soma de linhas/colunas, Diagonal Principal e Diagonal Secundária.	Grade Cartesiana 2D
Capítulo 12	Fase 6	**Structs (Estruturas Heterogêneas): Modelando o Mundo Real** Criação de tipos compostos (registros); acesso via operador ponto (`.`); uso de `typedef` e construção de sistemas completos com Vetores de Structs.	Ficha Cadastral Agrupada
Capítulo 13	Revisão Geral	**Pensando como um Programador: Guia de Depuração e Boas Práticas** Decomposição em 4 passos no papel; dicionário tradutor das mensagens reais do compilador GCC e checklist pré-entrega.	Método Investigativo
Capítulo 14	Bibliografia	**Referências Bibliográficas e Documentais (Padrão ABNT NBR 6023)** Livros-texto clássicos, artigos históricos originais com DOI, normas internacionais ISO/ANSI e documentação técnica utilizada na elaboração desta obra.	Fontes Primárias e Clássicas

ORIGENS DO PENSAMENTO ALGORÍTMICO

As Raízes da Computação: Dos Teares aos Laboratórios Bell

Como a indústria têxtil francesa, a matemática vitoriana e a criação do sistema UNIX deram origem à linguagem C.

Para compreender por que uma linguagem de programação funciona da maneira que funciona, vale a pena olhar para o problema humano que ela veio resolver. Antes de existirem telas, teclados ou microprocessadores de silício, a humanidade já enfrentava dois desafios antigos: como automatizar tarefas repetitivas sem erro humano e como registrar uma sequência de instruções para que uma máquina pudesse executá-la sozinha.

1.1 O Tear de Jacquard (1804) e o Nascimento do Estado Binário

No início do século XIX, na cidade de Lyon, na França, a produção de tecidos de seda com desenhos adamascados exigia dias de trabalho exaustivo. Crianças aprendizes precisavam levantar manualmente centenas de fios da urdidura a cada passada da lançadeira. Se um único fio fosse levantado na ordem errada, todo o desenho do tecido ficava comprometido.

Em 1804, o inventor francês **Joseph-Marie Jacquard** apresentou uma solução mecânica engenhosa: ele acoplou ao tear uma corrente de cartões de papelão grosso perfurados. Cada posição do cartão correspondia a uma agulha metálica que controlava um fio. Quando o cartão avançava:

- Se a agulha encontrasse um **furo no cartão**, ela atravessava o papelão e levantava o fio (estado ativo, equivalente ao bit `1`).
- Se a agulha encontrasse o **papelão fechado (sem furo)**, ela era barrada e o fio permanecia abaixado (estado inativo, equivalente ao bit `0`).

O impacto dessa invenção ultrapassou a indústria têxtil. Para mudar o desenho do tecido, não era mais necessário reconstruir a máquina de madeira; bastava trocar a fita de cartões perfurados. Pela primeira vez na história, havia uma separação clara entre a máquina física (o que hoje chamamos de *hardware*) e a sequência lógica de instruções gravadas nos cartões (o que hoje chamamos de *software*).

1.2 Charles Babbage e Ada Lovelace (1843): O Primeiro Algoritmo

Trinta anos depois, na Inglaterra, o matemático **Charles Babbage** estava inconformado com a quantidade de erros presentes nas tabelas matemáticas de navegação marítima, que eram calculadas à mão por calculistas humanos. Inspirado diretamente pelos cartões perfurados de Jacquard, Babbage projetou a **Máquina Analítica** (*Analytical Engine*), um computador mecânico movido a vapor que já possuía os mesmos componentes conceituais de um computador moderno: uma memória de engrenagens (chamada por ele de *Armazém* / *Store*) e um processador aritmético (chamado de *Moinho* / *Mill*).

Em 1842, o engenheiro italiano Luigi Menabrea publicou em francês um artigo descrevendo a máquina de Babbage. A matemática inglesa **Ada Augusta King, Condessa de Lovelace**, assumiu a tarefa de traduzir o artigo para o inglês e adicionou sete extensas notas técnicas de sua própria autoria (identificadas pelas letras A a G), que acabaram ficando três vezes maiores que o artigo original.

Na célebre **Nota G (1843)**, Ada Lovelace detalhou passo a passo uma sequência de operações, variáveis temporárias e repetições condicionais para que a Máquina Analítica calculasse a sequência dos *Números de Bernoulli*. Mais do que escrever o **primeiro algoritmo de computador documentado da história**, Ada teve uma percepção filosófica que nem mesmo Babbage havia alcançado: ela escreveu que a máquina não estava limitada a fazer contas aritméticas; se os números representassem alturas de notas musicais ou símbolos lógicos, a máquina poderia compor música complexa ou processar qualquer tipo de informação estruturada.

LINHA DO TEMPO: OS QUATRO PILARES HISTÓRICOS DA PROGRAMAÇÃO

Da codificação física em cartões perfurados à criação da linguagem C nos Laboratórios Bell

1804

Tear de Jacquard

Joseph-Marie Jacquard

Cartões perfurados controlavam agulhas têxteis. Introduziu a lógica binária física (furo / sem furo) separando máquina e instrução.

1843

Primeiro Algoritmo

Ada Lovelace

Publicou na Nota G o algoritmo para os números de Bernoulli na Máquina Analítica, prevendo o uso de variáveis e laços de repetição.

1936

Máquina Universal

Alan Turing

Definiu matematicamente o que é um problema computável e provou que um único hardware pode executar qualquer programa gravado na memória.

1972

Linguagem C e UNIX

Dennis Ritchie & K. Thompson

Criada nos Bell Labs para reescrever o sistema operacional UNIX. Combinou portabilidade entre máquinas com acesso direto à memória.

Figura 1.1 — Evolução histórica dos conceitos que pavimentaram a arquitetura dos computadores modernos e da linguagem C.

1.3 Alan Turing (1936), os Primeiros Computadores Eletrônicos e o Transistor

Quase um século após Ada Lovelace, em novembro de 1936, o jovem matemático britânico **Alan Mathison Turing** publicou o artigo *"On Computable Numbers, with an Application to the Entscheidungsproblem"*. Nesse trabalho, Turing idealizou um dispositivo teórico simples: uma fita infinita dividida em células (contendo símbolos 0 ou 1) e um cabeçote de leitura e escrita capaz de se mover para a esquerda ou para a direita seguindo uma tabela de estados. Turing provou matematicamente que, se as próprias instruções do programa fossem codificadas como números e guardadas na mesma fita que os dados, uma única máquina — a **Máquina Universal de Turing** — seria capaz de simular qualquer outra máquina existente.

Durante a Segunda Guerra Mundial e nos anos seguintes, essas ideias saíram do papel. Surgiram o Colossus britânico, o ENIAC norte-americano (1945) e a arquitetura de programa armazenado formalizada por John von Neumann. Contudo, os primeiros computadores eletrônicos utilizavam milhares de válvulas termiônicas parecidas com lâmpadas incandescentes: elas esquentavam intensamente, consumiam a energia de um quarteirão inteiro e queimavam a cada poucas horas. A revolução silenciosa ocorreu em dezembro de 1947, dentro dos **Laboratórios Bell (Bell Labs)**, em Nova Jersey, quando John Bardeen, Walter Brattain e William Shockley inventaram o **transistor semicondutor**, permitindo substituir válvulas de vidro gigantes por chaves eletrônicas microscópicas de silício.

1.4 O Problema do Assembly e o Nascimento da Linguagem C (1969–1972)

Ainda nos Laboratórios Bell, entre 1969 e 1972, dois pesquisadores chamados **Ken Thompson** e **Dennis Ritchie** trabalhavam no desenvolvimento de um novo sistema operacional multitarefa batizado de **UNIX**. Naquela época, vigorava uma regra dura na computação: sistemas operacionais precisavam ser escritos inteiramente em linguagem *Assembly* (linguagem de montagem específica de cada fabricante de processador).

O problema do Assembly era a falta total de portabilidade. Se você escrevesse 20 mil linhas de código Assembly para um minicomputador DEC PDP-7 e a universidade comprasse no ano seguinte um computador DEC PDP-11, o código antigo não servia para nada: era necessário reescrever todo o sistema operacional do zero, pois cada processador possuía nomes de registradores e códigos de instrução diferentes.

Para escapar dessa armadilha, Ken Thompson criou inicialmente uma linguagem simplificada chamada **B** (inspirada na linguagem BCPL de Martin Richards, da Universidade de Cambridge). Porém, a linguagem B não possuía tipos de dados: ela tratava tudo na memória como uma única palavra genérica, o que impedia manipular caracteres individuais (bytes) ou números reais com eficiência no novo computador PDP-11.

Foi então que **Dennis Ritchie**, entre 1971 e 1972, projetou a sucessora natural da linguagem B — batizando-a simplesmente de **Linguagem C**. Ritchie adicionou à linguagem um sistema rigoroso de tipos de dados (`char` , `int` , `float` , `double`), estruturas de agrupamento (`struct`) e acesso direto a endereços de memória. Em 1973, Ritchie e Thompson fizeram algo inédito: reescreveram mais de 90% do núcleo (*kernel*) do sistema operacional UNIX utilizando a própria linguagem C. A partir daquele momento, para levar o UNIX para qualquer computador novo do mundo, bastava criar um compilador de C para aquela máquina.

Nível da Linguagem	Como a Instrução é Escrita	Portabilidade	Leitura Humana
1. Código de Máquina (Binário)	`10110000 00000101`	Nula (exclusivo de um único modelo de chip)	Impossível sem consultar manuais de fábrica
2. Assembly (Baixo Nível)	`MOV AL, 5` `ADD AL, BL`	Muito baixa (amarrado à família do processador)	Difícil e extensa (exige dezenas de linhas para um cálculo)
3. Linguagem C (Médio/Alto Nível)	`int soma = a + 5;`	**Universal** (compila em PCs, celulares, servidores e carros)	Clara, estruturada e próxima da notação algébrica

Por que C continua insubstituível mais de 50 anos depois? Quando você usa Python, saiba que o interpretador oficial do Python (chamado *CPython*) é escrito inteiramente em C. O sistema operacional Linux, o núcleo do Windows e do macOS, o sistema de controle de versão Git (criado por Linus Torvalds em C), os bancos de dados PostgreSQL, MySQL e SQLite, além dos sistemas de freios ABS, sondas da NASA e placas Arduino utilizam C ou C++. Aprender C não é estudar o passado: é abrir o capô do motor que move toda a tecnologia moderna.

DO TEXTO AO SILÍCIO

O que é uma Linguagem de Programação e Como o Compilador Funciona

Entenda como suas linhas de código em C são verificadas, traduzidas e transformadas em um programa executável.

No cotidiano humano, comunicamo-nos por meio de *linguagens naturais*, como o português ou o inglês. As linguagens naturais são ricas em figuras de linguagem, contexto implícito e ambiguidade. Se alguém disser a você: *"Vá ao mercado e traga um litro de leite; se tiver ovos, traga seis"*, você compreende pelo bom senso que deve trazer seis ovos, e não seis litros de leite.

Um computador eletrônico não possui bom senso, intuição nem capacidade de adivinhar intenções. Dentro do processador (CPU) e da memória RAM, tudo o que existe são milhões de transistores operando como interruptores elétricos: ou passa tensão elétrica (bit `1`), ou não passa tensão elétrica (bit `0`). Cada agrupamento de 8 bits forma **1 Byte**, capaz de representar $2^8 = 256$ padrões diferentes (de `00000000` a `11111111`).

Uma **Linguagem de Programação** é um idioma formal, dotado de vocabulário restrito (palavras reservadas) e regras gramaticais exatas (sintaxe), projetado para que um ser humano possa descrever algoritmos sem nenhuma ambiguidade. Mas, como o processador só entende sequências binárias em linguagem de máquina, precisamos de um programa tradutor que converta o nosso texto em C para instruções binárias da CPU.

2.1 As Duas Estratégias de Tradução: Compilação vs. Interpretação

Existem duas formas clássicas de traduzir uma linguagem de programação para a linguagem de máquina do processador: a **Compilação** e a **Interpretação**. Compreender essa diferença explica por que programas em C são tão velozes.

Característica	Linguagens Compiladas (Ex: C, C++, Rust, Go)	Linguagens Interpretadas (Ex: Python, JavaScript)
Como ocorre a tradução	Antes de o programa rodar, um software chamado **Compilador** lê o arquivo de código-fonte inteiro de ponta a ponta, verifica todos os erros e gera um arquivo binário final (executável).	Não gera um executável independente antes. Um programa chamado **Interpretador** lê uma linha do código-fonte, traduz na hora para a CPU, executa, e só então passa para a linha de baixo.
Analogia Prática	Traduzir um livro inteiro do inglês para o português, revisar e imprimir a edição traduzida. Na hora de ler, a leitura flui na velocidade máxima, sem pausas.	Assistir a uma palestra em outro idioma com um tradutor simultâneo ao lado: toda vez que você quiser ouvir a palestra novamente, o tradutor precisará estar presente traduzindo frase por frase.
Velocidade e Uso de Memória	**Desempenho máximo.** O programa já está em código nativo da CPU e consome o mínimo possível de memória RAM.	Mais lento (de 10 a 50 vezes em laços matemáticos puros), pois o interpretador consome tempo e memória traduzindo durante a execução.
Detecção de Erros de Sintaxe	Se houver um erro de sintaxe na linha 90, o compilador avisa imediatamente e **impede a geração do executável** até que você corrija.	O programa começa a rodar normalmente e só trava quando a execução alcança a linha defeituosa.

2.2 Por Dentro do GCC: As Quatro Etapas da Compilação em C

Ao longo deste curso, utilizamos o **GCC (GNU Compiler Collection)**, o compilador padrão mundial da linguagem C. Quando escrevemos um arquivo chamado `programa.c` e clicamos no botão *Run* do OnlineGDB (ou digitamos `gcc programa.c -o programa` no terminal do Linux/Windows), muita gente imagina que a tradução acontece em um passo único. Na realidade, o GCC executa uma linha de montagem composta por **quatro fases sequenciais**:

PIPELINE DE COMPILAÇÃO EM C (ARQUITETURA INTERNA DO GCC)

Os 4 processos que transformam o arquivo .c em um programa binário executável

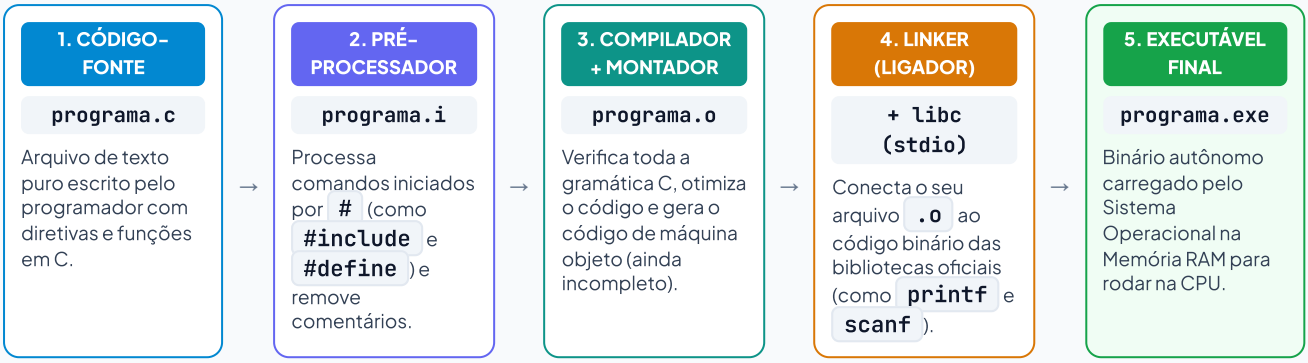


Figura 2.1 — Cadeia completa de compilação do GCC: cada etapa cumpre um papel específico e pode emitir alertas próprios.

1. **Etapa 1 — O Pré-Processador (`cpp`):** Antes que a análise da linguagem C comece, o pré-processador varre o seu texto procurando linhas que começam com o símbolo cerquilha (`#`), chamadas de *diretivas de pré-processamento*. Quando ele encontra `#include <stdio.h>` no topo do seu arquivo, ele literalmente copia e cola o conteúdo do arquivo de cabeçalho `stdio.h` (que contém as declarações das funções de entrada e saída) para dentro do seu código. Ele também apaga todos os seus comentários (`//`), pois comentários existem apenas para humanos lerem.
2. **Etapa 2 — O Compilador Propriamente Dito (`cc1`) e o Montador (`as`):** Esta é a etapa onde as regras da linguagem C são cobradas com rigor. O compilador analisa se todas as variáveis foram declaradas com seus tipos, se os parênteses e chaves foram fechados e se todas as instruções terminam com ponto-e-vírgula (`;`). Se houver qualquer violação gramatical, a compilação para aqui e exibe um **Erro de Sintaxe**. Caso tudo esteja correto, ele traduz o código C para instruções Assembly e o montador gera o chamado **Arquivo Objeto (`programa.o`)**.
3. **Etapa 3 — O Linker ou Ligador (`ld`):** O arquivo objeto `programa.o` já está em binário, mas ainda falta uma peça: quando você chamou `printf("Ola");` no seu código, o compilador sabia que a função `printf` existia (graças ao `stdio.h`), mas o código binário real que desenha as letras na tela mora dentro da Biblioteca Padrão do C (`libc`), já pré-compilada no computador. O papel do **Linker** é costurar o seu `programa.o` com o binário da biblioteca padrão, gerando o arquivo executável final.
4. **Etapa 4 — O Carregador (*Loader*) e a Execução na RAM:** Quando o executável fica pronto, o Sistema Operacional copia esse programa para dentro da **Memória RAM** e entrega o controle para o processador começar a executar a partir da primeira linha da função `main()`.

🔍 DIÁRIO DE DETETIVE: OS 3 TIPOS DE ERROS QUE VOCÊ ENCONTRARÁ NO CURSO

Quando algo dá errado, saber identificar em qual etapa o problema ocorreu economiza horas de estudo:

- **1. Erro de Compilação (Sintaxe):** Você esqueceu um `;`, escreveu `int x = ;` ou esqueceu de declarar uma variável. O GCC avisa o número da linha antes mesmo de o programa abrir.
- **2. Erro de Linkagem (*Linker Error*):** Você escreveu `prinft("Oi");` (trocando o *f* e o *t*). A gramática parece válida, mas na Etapa 4 o Linker procura a função `prinft` na biblioteca padrão, não a encontra e emite: `undefined reference to 'prinft'`.
- **3. Erro Lógico ou de Execução (*Runtime / Bug*):** O programa compila sem nenhum aviso e abre normalmente, mas você pediu para calcular a média somando `n1 + n2 / 2` sem parênteses, ou esqueceu o símbolo `&` no `scanf`. O compilador não lê mentes: se a frase estiver gramaticalmente correta em C, ele compila, mesmo que a fórmula matemática esteja errada!

FERRAMENTAS SEM BARREIRAS

O Ambiente de Desenvolvimento: Dominando o OnlineGDB

O que é uma IDE, por que escolhemos a computação em nuvem e como usar o editor, o terminal e o depurador visual.

Para programar em C, tecnicamente você precisaria apenas de dois programas separados: um editor de texto simples (como o Bloco de Notas ou o Vim) para escrever o arquivo `.c` e um compilador (como o GCC) acionado por comandos no terminal. No entanto, alternar o tempo todo entre janelas de texto e linhas de comando é cansativo e pouco produtivo, principalmente para quem está dando os primeiros passos.

Por isso, utilizamos o que a engenharia de software chama de **IDE (*Integrated Development Environment* — Ambiente de Desenvolvimento Integrado)**. Uma IDE reúne em uma única tela quatro ferramentas essenciais:

1. **Editor de Código-Fonte Inteligente:** Numera as linhas automaticamente, colore as palavras-chave da linguagem C (*syntax highlighting*) para facilitar a leitura e indenta os blocos de código.
2. **Botão de Compilação Automatizada:** Aciona todo o pipeline do GCC (Pré-processador, Compilador e Linker) com um único clique ou atalho de teclado.
3. **Console / Terminal Interativo Integrado:** Uma janela conectada ao seu programa onde aparecem as mensagens do `printf` e onde você digita os dados solicitados pelo `scanf`.
4. **Depurador Visual (*Debugger*):** Uma ferramenta que permite pausar o programa no meio da execução e inspecionar quanto vale cada variável na memória RAM passo a passo.

3.1 Por que Escolhemos o OnlineGDB em vez de IDEs Instaladas?

Tradicionalmente, cursos de programação em C pediam aos estudantes que instalassem em seus computadores pessoais softwares como *Dev-C++*, *Code::Blocks* ou *VS Code com MinGW*. Na prática de sala de aula e de extensão universitária, essa exigência sempre gerou três grandes problemas:

- **A Barreira da Configuração Inicial:** No Windows, instalar o compilador GCC via MinGW frequentemente exige configurar variáveis de ambiente do sistema (`PATH`). Muitos alunos passavam a primeira semana inteira sem conseguir compilar um único "Olá, Mundo" porque o instalador falhava ou porque o antivírus bloqueava o arquivo `.exe` recém-gerado.
- **Desigualdade de Hardware e Sistemas:** Nem todo estudante possui um notebook moderno em casa. Muitos estudam em computadores compartilhados da família, Chromebooks escolares, sistemas Linux ou até mesmo pelo navegador do celular no ônibus a caminho da universidade.
- **Conflito de Versões ("No meu PC funcionava!"):** IDEs antigas como o Dev-C++ trazem versões defasadas do compilador GCC de mais de dez anos atrás, que aceitam comportamentos fora do padrão atual. Quando o estudante levava o código para o laboratório da universidade, o programa apresentava erros inesperados.

Para eliminar 100% desses obstáculos e cumprir o pilar de acessibilidade universal dos **Recursos Educacionais Abertos (REA)**, adotamos o **OnlineGDB** (https://www.onlinegdb.com). No OnlineGDB, o seu navegador funciona apenas como uma janela visual: quando você clica em *Run*, o seu código é enviado para um servidor Linux de alta velocidade na nuvem equipado com o compilador GCC oficial, compilado e executado em frações de segundo.

ANATOMIA DA INTERFACE DO ONLINEGDB

Os quatro setores fundamentais que você utilizará em todas as aulas e listas de exercícios



1. Seletor Language: C (Canto Superior Direito)
Antes de digitar qualquer linha, abra a caixa *Language* e selecione **C**. Se ela ficar em branco ou em Python/C++, o código não compilará corretamente.

2. Botão Verde "Run" (ou Tecla F9)
Salva temporariamente seu arquivo `main.c`, aciona o compilador GCC no servidor e abre o console interativo na parte inferior da tela.

3. Editor Numerado com Auto-Fechamento
Quando o GCC indicar um erro como `main.c:6:5`, basta olhar o número `6` na margem esquerda do editor para encontrar a linha exata.

4. Console Interativo e Botão "Stop"
Sempre que o programa estiver aguardando um `scanf`, o cursor piscará no painel preto inferior. Se criar um laço infinito sem querer, clique no botão vermelho **Stop**!

Figura 3.1 — Interface de trabalho do OnlineGDB: edição numerada no topo, seletor de linguagem à direita e terminal interativo na base.

3.2 O Superpoder Escondido no OnlineGDB: Como Usar o Botão "Debug"

O próprio nome *OnlineGDB* vem do **GDB (GNU Project Debugger)**, o depurador oficial do projeto GNU criado por Richard Stallman. Muitos estudantes usam apenas o botão verde *Run*, mas aprender a usar o botão azul **Debug (F8)** transforma completamente a sua compreensão de programação, pois ele permite enxergar a memória RAM funcionando em câmera lenta:

1. **Marque um Ponto de Parada (*Breakpoint*):** No editor de código, clique com o mouse exatamente à esquerda do número da linha onde você quer começar a investigar (por exemplo, na linha 5). Aparecerá uma **bolinha vermelha** ao lado do número.
2. **Inicie o Modo Debug:** Clique no botão **Debug (F8)**. O programa vai começar a rodar e ficará congelado exatamente na linha da bolinha vermelha, destacando-a em verde.
3. **Avance Linha por Linha (*Step Over*):** No painel que se abre à direita, clique em **Next / Step Over**. O computador executará apenas aquela linha e parará na linha seguinte.
4. **Observe a Tabela *Local Variables*:** No canto inferior direito, o OnlineGDB mostra uma tabela com o nome de todas as suas variáveis e o valor exato guardado dentro de cada gaveta da memória naquele instante! Assim que você executa a linha do `scanf` ou de uma soma dentro de um laço `for`, você vê o número mudando na tabela diante dos seus olhos.

Como salvar e compartilhar seus exercícios no OnlineGDB: Para guardar o seu código no computador e enviar na plataforma da disciplina, clique no botão laranja **Download Code** (ícone de seta para baixo na barra superior) — ele baixará o arquivo `main.c`. Já se você estiver com dúvida em um exercício e quiser pedir ajuda a um colega, monitor ou professor, clique no botão laranja **Share**: ele cria um link exclusivo daquele código para enviar por mensagem!

PRIMEIROS PASSOS NO CÓDIGO

Anatomia Completa de um Programa em C

Dessecando cada símbolo, diretiva, chave, comentário e sequência de escape do esqueleto padrão da linguagem C.

Em 1978, quando Brian Kernighan e Dennis Ritchie publicaram a primeira edição do livro *The C Programming Language*, o primeiro exemplo apresentado nas páginas iniciais era um pequeno programa cuja única tarefa consistia em escrever as palavras `hello, world` na tela do terminal. Desde então, criar o programa "Olá, Mundo!" tornou-se o ritual de passagem de todo estudante de computação.

Por trás de apenas seis ou sete linhas de código, porém, já estão presentes todas as regras estruturais que governam desde um exercício de sala de aula até um sistema operacional de milhões de linhas. Vamos examinar um exemplo ligeiramente expandido e dissecar o papel exato de cada linha:

ANATOMIA_PADRAO.C

```c
/* ============================================================
 * Programa: Saudacao e Boas-Vindas ao Curso
 * Autoria:  Equipe Try, Catch & Learn (UTFPR-CP)
 * ============================================================ */
#include <stdio.h>

int main(void) {
    // Exibe o cabecalho alinhado com tabulacoes e quebras de linha:
    printf("========================================\n");
    printf("\tPROJETO TRY, CATCH & LEARN\n");
    printf("========================================\n");
    printf("Bem-vindo ao laboratorio de Linguagem C!\n");

    return 0;
}
```

SAÍDA NO CONSOLE DO ONLINEGDB

```
========================================
        PROJETO TRY, CATCH & LEARN
========================================
Bem-vindo ao laboratorio de Linguagem C!
...Program finished with exit code 0
```

4.1 Raio-X Linha por Linha: O Papel de Cada Elemento

1. **Os Comentários (`/* ... */` e `//`):** Tudo o que está escrito entre `/*` e `*/` (comentário de múltiplas linhas, padrão ANSI C89) ou após duas barras paralelas `//` até o final da linha (comentário de linha única, padronizado no C99) é **completamente ignorado pelo compilador**. Comentários servem para documentar o raciocínio humano: explicar qual fórmula está sendo aplicada, quem escreveu o arquivo ou o que significa determinada variável.
2. **A Diretiva `#include <stdio.h>` :** Como vimos no Capítulo 2, linhas iniciadas por `#` são ordens para o Pré-Processador. O núcleo da linguagem C não traz embutido nenhum comando de teclado ou monitor. Essas ferramentas estão guardadas na biblioteca padrão de entrada e saída, cujo catálogo chama-se `stdio.h` (sigla do inglês *Standard Input/Output Header*). Sem essa linha no topo, o compilador não reconhecerá os comandos `printf` e `scanf` . Note que diretivas `#include` **nunca** levam ponto-e-vírgula no final!
3. **O Ponto de Partida: `int main(void)` :** Um programa grande pode ter dezenas de funções espalhadas pelo arquivo, mas o Sistema Operacional precisa saber exatamente por onde começar a execução. Na linguagem C, a regra é absoluta: todo programa começa na função chamada `main` . A palavra `int` antes do nome indica que, ao encerrar sua execução, a função `main` devolverá um número inteiro para o Sistema Operacional. A palavra `void` (ou os parênteses vazios `()`) indica que ela não recebe parâmetros externos na abertura.
4. **As Chaves Delimitadoras de Bloco `{` e `}` :** As chaves funcionam como as paredes de uma sala: a chave de abertura `{` marca onde começa o corpo da função `main` , e a chave de fechamento `}` marca onde ele termina. Para manter o código organizado e fácil de ler, adotamos o recuo visual (chamado de **indentação**, geralmente de 4 espaços ou 1

tecla `Tab`) para todas as linhas que estão dentro de um par de chaves.

5. **O Ponto-e-Vírgula (`;`) como Finalizador de Instrução:** Para o compilador C, quebras de linha com a tecla Enter são apenas espaços em branco. O que realmente informa ao compilador que uma instrução terminou é o **ponto-e-vírgula (`;`)**. Ele equivale ao ponto final de uma frase na língua portuguesa.
6. **O Comando de Encerramento `return 0;`:** É a última instrução da função `main`. Quando o processador chega a esta linha, o programa termina imediatamente e devolve o código numérico `0` para o Sistema Operacional (Windows ou Linux). Por convenção universal criada no sistema UNIX, o código `0` significa *"execução concluída com 100% de sucesso"*, enquanto números diferentes de zero indicam falhas.

4.2 As Sequências de Escape dentro do `printf`

Se você colocar dois comandos `printf("Ola");` e `printf("Mundo");` um embaixo do outro no código, terá uma surpresa ao executar: eles aparecerão grudados na mesma linha da tela (`OlaMundo`). Isso acontece porque o `printf` não pula de linha sozinho. Para controlar o cursor da tela, utilizamos códigos especiais iniciados pela **barra invertida (`\`)**, chamados de *Sequências de Escape*:

Sequência	Nome Técnico	Efeito Visual no Terminal
`\n`	Newline (Nova Linha)	Quebra a linha imediatamente e posiciona o cursor no início da linha de baixo.
`\t`	Horizontal Tab (Tabulação)	Insere um espaçamento horizontal alinhado (equivalente à tecla Tab), excelente para alinhar colunas de tabelas no terminal.
`\"`	Aspas Duplas Literais	Permite imprimir o símbolo de aspas `"` na tela sem que o compilador pense que o texto do `printf` acabou (ex: `printf("Linguagem \"C\"");`).
`\\`	Barra Invertida Literal	Imprime uma única barra invertida `\` na tela (necessário ao exibir caminhos de pastas como `C:\\Users`).
`\0`	Caractere Nulo (Null Byte)	Marca invisível de valor zero que indica o final de um texto (*string*) na memória RAM.

Atenção à Regra do Case Sensitivity (Maiúsculas vs. Minúsculas): A linguagem C diferencia rigorosamente letras minúsculas de letras maiúsculas. Para o compilador, `main`, `Main` e `MAIN` são três palavras completamente diferentes. Todas as palavras reservadas da linguagem C (`int`, `float`, `return`, `if`, `while`, `for`) e as funções padrão (`printf`, `scanf`) devem ser escritas **sempre em letras minúsculas**!

🔍 DIÁRIO DE DETETIVE: DECIFRANDO OS 3 ERROS MAIS COMUNS DA PRIMEIRA AULA

- `error: expected ';' before 'return'`: O compilador sentiu falta de um ponto-e-vírgula logo antes da linha do `return`. Olhe sempre para o final da **linha imediatamente anterior** à linha apontada pelo erro!
- `warning: implicit declaration of function 'printf'`: Você usou o `printf`, mas esqueceu de colocar `#include <stdio.h>` na primeira linha do arquivo, ou escreveu `printf` com erro de digitação.
- `error: expected declaration or statement at end of input`: Você abriu uma chave `{` no início da função, mas esqueceu de fechar a última chave `}` no final do arquivo.

O MODELO FÍSICO DAS GAVETAS

Memória RAM, Variáveis, Tipos Primitivos e Operadores

Como o computador aloca espaço físico na memória, armazena diferentes tipos de dados e executa cálculos matemáticos.

Um programa que apenas imprime mensagens fixas na tela é como um cartaz estático. Para que um software resolva problemas reais — como calcular a média semestral de um estudante, simular a física de um jogo ou controlar o saldo de uma conta bancária —, ele precisa guardar informações temporárias, combiná-las em expressões matemáticas e atualizar seus valores ao longo do tempo.

Toda informação que um programa manipula enquanto está aberto fica hospedada na **Memória RAM (*Random Access Memory* — Memória de Acesso Aleatório)**. Diferentemente do disco rígido ou SSD (onde seus arquivos ficam salvos permanentemente mesmo quando o computador é desligado), a Memória RAM é uma memória ultrarrápida e *volátil*: ela funciona como a mesa de trabalho do processador enquanto o programa está rodando, e é esvaziada assim que o programa termina.

5.1 A Metáfora das Gavetas e as 4 Propriedades de uma Variável

Para visualizar a Memória RAM sem mistério, imagine um gigantesco armário de correio composto por bilhões de pequenas células numeradas sequencialmente em código hexadecimal (como `0x7ffe00`, `0x7ffe01`, `0x7ffe02`...). Cada célula individual tem o tamanho exato de **1 Byte (8 bits)**.

Seria desumano programar dizendo ao computador: *"guarde o número 20 nos 4 bytes que começam no endereço hexadecimal 0x7ffe01"*. Para que não precisemos decorar códigos numéricos de endereços, a linguagem C nos permite dar um **apelido simbólico** para esse espaço de memória. Esse espaço reservado e etiquetado recebe o nome de **Variável**.

Na linguagem C, toda variável existente no seu programa possui **quatro propriedades físicas inseparáveis**:

1. **Endereço Físico na RAM (Acessado com `&`):** O número de série da gaveta no hardware onde os bytes começam (ex: `0x7ffe01`). É fixo enquanto a variável existir.
2. **Identificador (Nome da Variável):** A etiqueta legível que nós colamos na frente da gaveta para nos referirmos a ela no código (ex: `idade`, `altura`, `salario`).
3. **Tipo de Dado (*Data Type*):** Define o tamanho da gaveta (quantos bytes de largura ela ocupa na RAM) e como os zeros e uns lá dentro devem ser interpretados (se como número inteiro, número com vírgula ou caractere de texto).
4. **Valor (Conteúdo):** O dado propriamente dito que está guardado dentro da gaveta naquele instante (e que pode variar ao longo do programa — daí o nome *variável*).

ANATOMIA DAS VARIÁVEIS NA MEMÓRIA RAM

Visualização das 4 propriedades: Endereço Físico (&), Nome (Identificador), Tipo (Tamanho) e Valor

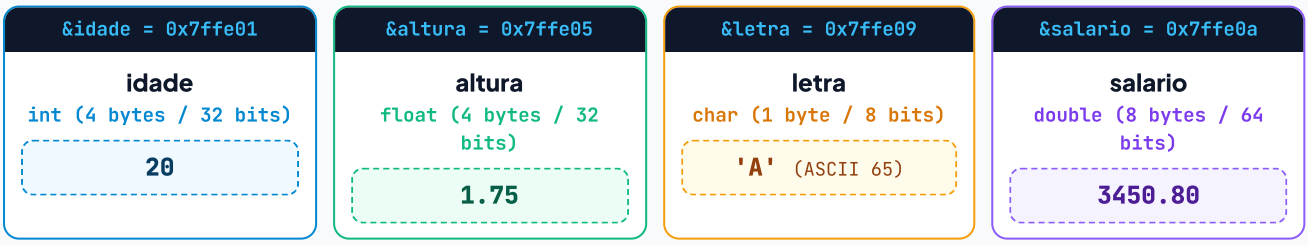


Figura 5.1 — Repare nos endereços no topo: como 'idade' ocupa 4 bytes (01, 02, 03 e 04), a variável seguinte ('altura') começa no byte 05!

5.2 Os Quatro Tipos Primitivos da Linguagem C

Como a linguagem C é uma linguagem de **tipagem estática e explícita**, você é obrigado a informar o tipo de cada variável no momento em que a cria (declaração). Isso permite ao compilador saber exatamente quantos bytes reservar na memória:

Tipo	Tamanho Típico	Faixa de Valores e Finalidade	Declaração em C	Marcador I/O
`int`	4 bytes (32 bits)	Números inteiros (positivos, negativos ou zero), sem parte fracionária. Vai de -2.147.483.648 até +2.147.483.647.	`int quantidade = 15;`	`%d` ou `%i`
`float`	4 bytes (32 bits)	Números reais (ponto flutuante de precisão simples). Possui cerca de 6 a 7 dígitos significativos de precisão após o ponto.	`float media = 7.85;`	`%f`
`double`	8 bytes (64 bits)	Números reais de **dupla precisão**. Ocupa o dobro de espaço do `float`, garantindo até 15 dígitos decimais exatos (ideal para engenharia e finanças).	`double pi = 3.14159265;`	`%lf` (no scanf)
`char`	1 byte (8 bits)	Armazena um único caractere delimitado por **aspas simples** (`'A'`, `'7'`, `'@'`). Internamente, guarda o código numérico inteiro da Tabela ASCII (ex: `'A'` vale `65`).	`char turma = 'B';`	`%c`

Regra de Ouro da Pontuação Decimal em Código: Em qualquer linguagem de programação, o separador de casas decimais é o **ponto americano (`.`)**, e nunca a vírgula! Escreva sempre `float altura = 1.75;`. Na linguagem C, a vírgula serve apenas para separar itens em uma lista (como declarar duas variáveis na mesma linha: `int a = 1, b = 2;`).

5.3 Regras para Nomear Variáveis e o Perigo do "Lixo de Memória"

Ao escolher o nome (identificador) de uma variável em C, o compilador exige o cumprimento de quatro regras:

- **Regra 1 (Caracteres Permitidos):** O nome pode conter apenas letras sem acento (`a-z`, `A-Z`), dígitos numéricos (`0-9`) e o símbolo sublinhado / *underline* (`_`). Espaços em branco, cedilha (`ç`), acentos (`~`, `´`) ou símbolos especiais (`@`, `-`, `$`) causam erro imediato de compilação.
- **Regra 2 (Proibido Começar com Número):** O primeiro caractere do nome deve ser obrigatoriamente uma letra ou `_`. Assim, `nota1` é válido, mas `1nota` é proibido.
- **Regra 3 (Palavras Reservadas):** Você não pode batizar uma variável com palavras que já pertencem à gramática do C (como `int`, `float`, `return`, `if`, `for`, `while`, `struct`).
- **Regra 4 (Clareza Semântica):** Prefira nomes que expliquem o significado do dado, usando o padrão *snake_case* (`media_final`) ou *camelCase* (`mediaFinal`) em vez de letras soltas como `x1`, `x2`, `aux`.

O Perigo Silencioso do "Lixo de Memória" (*Memory Garbage*): Quando você escreve apenas `int soma;` sem atribuir um valor inicial, muitos iniciantes acham que a gaveta começa automaticamente valendo zero. **Na linguagem C, isso é falso!** Para ser ultrarrápido, o C apenas reserva os 4 bytes na RAM sem limpá-los. Isso significa que dentro da variável `soma` estarão os bits velhos que sobraram de outros programas que usaram aquele endereço antes (podendo valer `-842150451` ou `32767`). Se você fizer `soma = soma + 10;` sem antes ter inicializado `int soma = 0;`, o resultado final será completamente absurdo!

5.4 Operadores Aritméticos, Precedência e a Pegadinha da Divisão

Na linguagem C, o símbolo de igualdade simples (`=`) não significa "é igual a", mas sim **Recebe (Operador de Atribuição)**. O computador resolve primeiro toda a conta que está à **direita** do `=` e, ao final, guarda o resultado dentro da variável que está à **esquerda**.

Operador	Operação Matemática	Exemplo Prático em C	Observação Importante
`+` e `-`	Adição e Subtração	`lucro = receita - custo;`	Menor prioridade na expressão (executados por último).
`*`	Multiplicação	`area = base * altura;`	Usa-se sempre o asterisco `*` (nunca `x` ou ponto).
`/`	Divisão	`metade = valor / 2.0;`	**Cuidado:** se ambos os lados forem `int`, corta as casas decimais!

Operador	Operação Matemática	Exemplo Prático em C	Observação Importante
`%`	Módulo (Resto da Divisão Inteira)	`resto = 17 % 5;` (Resulta em 2)	Exclusivo para números inteiros (`int`). Se `n % 2 == 0` , o número `n` é **par**!

Assim como na matemática escolar, a linguagem C respeita a **Hierarquia de Precedência**: expressões dentro de parênteses `( )` são calculadas primeiro; em seguida vêm multiplicação (`*`), divisão (`/`) e resto (`%`); e, por último, adição (`+`) e subtração (`-`). Portanto, para calcular a média entre duas notas, você deve obrigatoriamente usar parênteses: `media = (n1 + n2) / 2.0;` . Se escrever `n1 + n2 / 2.0` , o C dividirá apenas a segunda nota por dois antes de somar!

DIVISAO_E_CASTING.C

```c
#include <stdio.h>

int main(void) {
    int pontos = 9, partidas = 2;

    // ARMADILHA: int dividido por int gera um int (9 / 2 vira 4, perdendo o .5!)
    float media_errada = pontos / partidas;

    // SOLUÇÃO (Type Casting): converte 'pontos' temporariamente para float antes de dividir
    float media_correta = (float) pontos / partidas;

    printf("Sem casting (truncada): %.2f\n", media_errada);
    printf("Com casting (exata):    %.2f\n", media_correta);
    return 0;
}
```

SAÍDA NO TERMINAL

```
Sem casting (truncada): 4.00
Com casting (exata):    4.50
```

DIÁLOGO COM O USUÁRIO

Entrada e Saída de Dados: printf, scanf e o Buffer do Teclado

Como formatar exibições na tela, por que o operador & é indispensável no scanf e como dominar a fila de entrada do teclado.

Todo algoritmo clássico segue o fluxo **Entrada → Processamento → Saída**. Na linguagem C, a dupla responsável por conectar o mundo externo (teclado e monitor) às gavetas da Memória RAM é formada pelas funções `printf` (saída formatada) e `scanf` (leitura formatada), ambas pertencentes à biblioteca `<stdio.h>`.

6.1 Dominando a Formatação de Saída com `printf`

Quando queremos exibir o conteúdo de uma ou mais variáveis no meio de uma frase, colocamos dentro das aspas do `printf` os **especificadores de formato** (`%d` para inteiros, `%f` para reais, `%c` para caractere único, `%s` para textos). Após fechar as aspas e colocar uma vírgula, listamos os nomes das variáveis na exata ordem em que os símbolos `%` apareceram no texto.

Por padrão, quando você imprime um número `float` ou `double` usando apenas `%f`, a linguagem C exibe **seis casas decimais** (por exemplo, `19.500000`). Para controlar a quantidade de casas após o ponto, colocamos um ponto e o número de casas desejadas entre o `%` e o `f` :

- `%.2f` — Arredonda a exibição para exatamente **2 casas decimais** (padrão para valores em Reais `R$` e médias escolares).
- `%.1f` ou `%.3f` — Exibe 1 ou 3 casas decimais, respectivamente.
- `%04d` — Exibe um número inteiro ocupando no mínimo 4 espaços, preenchendo com zeros à esquerda (ex: o número `7` aparece como `0007`).

6.2 A Leitura com scanf e o Papel Físico do Operador &

Enquanto o `printf` envia dados da memória para a tela, o `scanf` faz o caminho inverso: ele pausa o programa, espera o usuário digitar um valor no teclado e apertar **Enter**, converte os caracteres digitados para o formato indicado e grava o valor dentro da variável na Memória RAM.

Aqui surge a dúvida mais importante da Fase 1: *Por que no* `printf("%d", idade);` *escrevemos apenas o nome da variável, mas no* `scanf("%d", &idade);` *somos obrigados a colocar o símbolo* **E comercial (`&`)** *antes do nome?*

POR QUE O SCANF EXIGE O SÍMBOLO & (OPERADOR DE ENDEREÇO)?

Comparação física entre ler uma cópia do valor (printf) e entregar uma encomenda no endereço da gaveta (scanf)

1. COMO FUNCIONA O PRINTF("...", IDADE)

```
printf("Idade: %d", idade); // Passa só a
cópia (20)
```

O `printf` **não altera** nenhuma variável. Ele apenas recebe uma fotocópia do número `20` que estava guardado em `idade` e desenha esse número no monitor. Ele não precisa saber onde a gaveta mora na RAM.

2. COMO FUNCIONA O SCANF("...", &IDADE)

```
scanf("%d", &idade); // Entrega o endereço
0x7ffe01!
```

O `scanf` é um **carteiro** que precisa depositar o número digitado dentro da gaveta original na RAM. O símbolo `&` significa *"Endereço físico de..."*. Sem o `&`, o carteiro não tem o endereço da casa e o programa trava (*Segmentation Fault*)!

Figura 6.1 — Diferença fundamental entre enviar o valor armazenado (printf) e enviar a coordenada hexadecimal da memória (& no scanf).

6.3 Entendendo o Buffer do Teclado (`stdin`) e o Bug do `char`

Quando você digita no teclado, as letras e números não vão diretamente para dentro da variável: eles entram primeiro em uma fila temporária de memória chamada **Buffer de Entrada Padrão (`stdin`)**.

Imagine que o seu programa pede a idade (`int`) e, logo na linha de baixo, pede a inicial do nome (`char`). Você digita `20` e aperta a tecla **Enter**. Nesse exato momento, entram três caracteres na fila do teclado: o dígito `'2'` , o dígito `'0'` e o caractere invisível da tecla Enter, que é o `'\n'` :

- O primeiro comando, `scanf("%d", &idade);`, consome os dígitos `20` e para assim que vê o `'\n'`, deixando o `'\n'` esquecido dentro da fila do teclado!
- Quando o programa chega ao segundo comando, `scanf("%c", &letra);`, o especificador `%c` lê qualquer caractere — inclusive espaços e quebras de linha! Como o `'\n'` do Enter anterior ainda estava esperando na fila, o `scanf("%c", ...)` engole esse `'\n'` na hora e passa direto sem deixar o usuário digitar a letra!

CADASTRO_PACIENTE_IMC.C

```c
#include <stdio.h>

int main(void) {
    int idade;
    float peso, altura, imc;
    char tipo_sanguineo;

    printf("Digite sua idade: ");
    scanf("%d", &idade);

    printf("Digite seu peso (kg) e sua altura (m): ");
    scanf("%f %f", &peso, &altura); // Lê dois floats separados por espaço ou Enter

    // REPARO DO BUFFER: O espaço em branco antes de %c (" %c") descarta o Enter (
) anterior!
    printf("Digite a letra do seu tipo sanguineo (A, B, O): ");
    scanf(" %c", &tipo_sanguineo);

    imc = peso / (altura * altura);

    printf("\n--- FICHA DE AVALIACAO ---\n");
    printf("Idade: %d anos | Sangue: Tipo %c | IMC Calculado: %.2f\n", idade, tipo_sanguineo, imc);

    return 0;
}
```

Duas Regras de Sobrevivência com o `scanf`:
1. **Nunca coloque textos ou `\n` dentro das aspas do `scanf`!** Se você escrever `scanf("Idade: %d\n", &idade);`, o programa vai travar esperando que o usuário digite literalmente a palavra "Idade: " no teclado. Dentro do `scanf` vão apenas os marcadores (`"%d"`, `"%f"`, `" %c"`).
2. **Sempre dê um espaço antes do `%c`:** Escrever `scanf(" %c", &variavel);` com aquele espaço inicial resolve 100% dos problemas de pulo de leitura de caracteres!

O PROGRAMA APRENDE A DECIDIR

Estruturas de Decisão: Condicionais if, else e switch-case

Dotando seus algoritmos de raciocínio lógico através de expressões relacionais, conectivos booleanos e desvios de fluxo.

Até o Capítulo 6, todos os nossos programas tinham um comportamento estritamente linear: eles começavam na primeira linha da função `main()` e executavam todas as instruções seguintes, uma por uma, sem jamais pular nenhuma linha. Na vida real, entretanto, qualquer processo exige tomar decisões baseadas em condições: um caixa eletrônico só libera o saque se o valor solicitado for menor ou igual ao saldo disponível; um aluno só é aprovado diretamente se sua média for maior ou igual a 7,0 e sua frequência for de pelo menos 75%.

Na linguagem C original (padrão C89), não existia um tipo primitivo separado para guardar "Verdadeiro" ou "Falso". Em vez disso, Dennis Ritchie definiu uma regra numérica simples e direta que vale para toda condição em C:

- O valor inteiro `0` **(zero)** representa **FALSO (*False*)**.
- Qualquer valor **diferente de zero** (geralmente o número `1`) representa **VERDADEIRO (*True*)**.

FLUXO DE DECISÃO CONDICIONAL COMPOSTA (IF / ELSE)

O teste lógico no losango funciona como uma agulha de trilho de trem: apenas um dos dois caminhos será percorrido

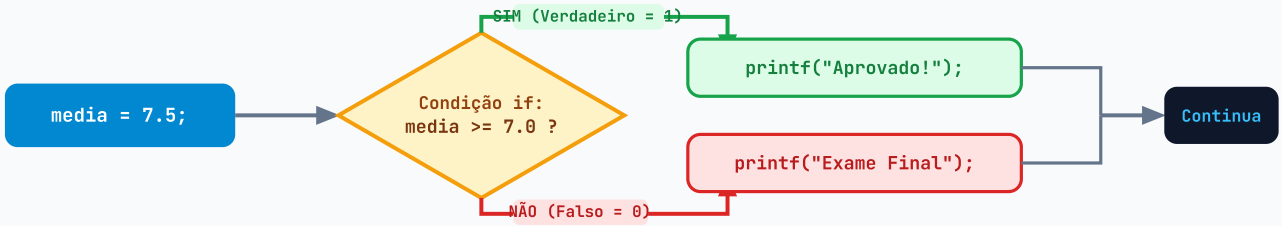


Figura 7.1 — Avaliação condicional: se a condição for verdadeira, o bloco else é inteiramente ignorado pelo processador.

7.1 Operadores Relacionais e Conectivos Lógicos (Álgebra de Boole)

Para construir as perguntas dentro dos parênteses do `if`, utilizamos os seis **Operadores Relacionais** para comparar valores e os três **Operadores Lógicos** (baseados na álgebra de George Boole) para combinar duas ou mais perguntas simultaneamente:

Categoria	Símbolo	Significado e Regra de Avaliação	Exemplo em C
Relacionais Comparam dois valores	`==`	**Igual a:** Resulta em `1` se os dois lados tiverem o mesmo valor. (Nunca confunda com `=` de atribuição!)	`if (resto == 0)`
	`!=`	**Diferente de:** Resulta em `1` se os valores forem distintos.	`if (divisor != 0)`
	`>` e `<`	**Maior que / Menor que** (estritos: não incluem o próprio limite).	`if (temperatura > 37.5)`
	`>=` e `<=`	**Maior ou igual a / Menor ou igual a** (incluem o valor limite).	`if (idade >= 18)`
Lógicos Unem condições	`&&` (E / AND)	**Conjunção Rigorosa:** Só resulta em Verdadeiro se a Condição 1 **E** a Condição 2 forem ambas verdadeiras.	`if (media >= 7.0 && faltas <= 15)`
	`||` (OU / OR)	**Disjunção Inclusiva:** Resulta em Verdadeiro se **pelo menos uma** das condições for verdadeira.	`if (dia == 1 || dia == 7)`
	`!` (NÃO / NOT)	**Negação Inversora:** Inverte o resultado lógico (transforma Verdadeiro em Falso e vice-versa).	`if (!(idade >= 18))`

7.2 Condicionais Encadeadas (`if / else if / else`) na Prática

Quando um problema possui apenas dois caminhos (ex: Par ou Ímpar), usamos um único par `if / else`. Porém, na maioria dos problemas reais existem três, quatro ou mais faixas mutuamente exclusivas — por exemplo, classificar a situação acadêmica de um aluno em *Aprovado Direto* (≥ 7,0), *Em Exame Final* (≥ 4,0 e < 7,0) ou *Reprovado Direto* (< 4,0). Para isso, encadeamos condições com `else if`:

SISTEMA_AVALIACAO_UTFPR.C

```c
#include <stdio.h>

int main(void) {
    float n1, n2, media;
    int frequencia;

    printf("Digite as duas notas do aluno (0 a 10): ");
    scanf("%f %f", &n1, &n2);
    printf("Digite a porcentagem de presenca (0 a 100): ");
    scanf("%d", &frequencia);

    media = (n1 + n2) / 2.0;
    printf("\nMedia Calculada: %.2f | Frequencia: %d%%\n", media, frequencia);

    // 1° Filtro: Verifica se reprovou por falta (frequência abaixo de 75%)
    if (frequencia < 75) {
        printf("Situacao: REPROVADO POR FALTA (minimo exigido: 75%%).\n");
    }
    // 2° Filtro: Se chegou aqui, tem presença suficiente! Avaliamos a nota:
    else if (media >= 7.0) {
        printf("Situacao: APROVADO DIRETO! Parabens pelo desempenho!\n");
    }
    else if (media >= 4.0) {
        float nota_necessaria = 10.0 - media;
        printf("Situacao: EXAME FINAL. Precisa tirar %.2f no exame.\n", nota_necessaria);
    }
    else {
        printf("Situacao: REPROVADO POR NOTA (media inferior a 4.0).\n");
    }

    return 0;
}
```

Duas Armadilhas Clássicas no `if` que Todo Iniciante Deve Evitar:
1. **Usar apenas um igual (`if (x = 5)`) em vez de dois (`if (x == 5)`):** Se você escrever `if (x = 5)`, o C não vai comparar x com 5; ele vai *guardar* o número 5 dentro de x e, como 5 é diferente de zero, o computador considerará a condição **sempre verdadeira**!
2. **Colocar ponto-e-vírgula logo após o parêntese do `if`:** Nunca escreva `if (media >= 7.0); { ... }`. Aquele ponto-e-vírgula encerra o `if` ali mesmo como uma instrução vazia, fazendo com que o bloco entre chaves abaixo seja executado sempre, mesmo que a nota seja zero!

7.3 Seleção Múltipla com `switch-case`

Quando precisamos testar uma única variável do tipo **inteiro (`int`)** ou **caractere (`char`)** contra vários valores exatos e constantes — como as opções de um menu de restaurante, códigos de operações de uma calculadora (`'+'`, `'-'`, `'*'`, `'/'`) ou dias da semana (1 a 7) —, a estrutura `switch-case` deixa o código muito mais limpo do que uma longa escada de `else if`:

```c
switch (operador) {
    case '+':
        printf("Resultado da Soma = %.2f\n", a + b);
        break; // O break encerra o switch e impede que ele desça para o caso de baixo!
    case '-':
        printf("Resultado da Subtracao = %.2f\n", a - b);
        break;
    default: // Opcional: executado se nenhum dos cases acima corresponder
        printf("Erro: Operador matematico desconhecido!\n");
}
```

Lembre-se de que o comando `switch` **não aceita variáveis reais (** `float` **/** `double` **) nem intervalos relacionais** (como `case > 10:`). Para intervalos numéricos ou números com vírgula, utilize sempre `if / else if / else`.

REPITA COMIGO: ADEUS TRABALHO BRAÇAL

Estruturas de Repetição: while, do-while, for e Teste de Mesa

Como automatizar tarefas repetitivas com precisão, controlar contadores e acumuladores e rastrear variáveis volta por volta.

Imagine que um professor peça a você um programa em C para ler as notas de 3 alunos e calcular a média da turma. Com o que aprendemos até aqui, você poderia declarar `n1, n2, n3` e usar três comandos `scanf`. Mas e se a turma tiver 60 alunos? E se for o sistema do ENEM processando a nota de 4 milhões de candidatos? Copiar e colar milhões de linhas de código seria inviável.

É aqui que entra a maior virtude dos computadores: a capacidade de executar um mesmo bloco de instruções milhares ou milhões de vezes através das **Estruturas de Repetição** (também chamadas de *Laços* ou *Loops*). Todo laço de repetição baseia-se em uma **condição de parada**: enquanto a condição permanecer verdadeira, o bloco se repete; assim que ela se torna falsa, o laço termina e o programa segue em frente.

8.1 Laço `while` (Enquanto) vs. Laço `do-while` (Faça... Enquanto)

Os laços `while` e `do-while` são indicados especialmente para situações de **repetição indeterminada** — isto é, quando não sabemos de antemão quantas vezes o usuário vai repetir a ação (por exemplo: *"continue pedindo a nota enquanto o usuário digitar um valor inválido menor que 0 ou maior que 10"*, ou *"continue somando preços até que o caixa digite 0 para encerrar a compra"*).

Critério	Laço `while (condicao) { ... }`	Laço `do { ... } while (condicao);`
Onde fica o teste lógico?	**No início (Pré-Teste):** Verifica a condição *antes* de entrar no bloco pela primeira vez.	**No final (Pós-Teste):** Executa o bloco primeiro e só verifica a condição ao final da volta.
Quantidade mínima de execuções	**Zero vezes:** Se a condição já começar falsa logo de cara, o computador pula o laço inteiro sem executar nenhuma vez.	**Uma vez garantida:** Mesmo que a condição seja falsa, o corpo é executado pelo menos na primeira passagem.
Detalhe de Sintaxe	Não leva ponto-e-vírgula após os parênteses da condição.	Exige **obrigatoriamente um ponto-e-vírgula** `( ; )` após o `while (condicao);` no fechamento!
Caso de Uso Clássico	Leitura com valor sentinela (ex: ler idades enquanto `idade != -1`).	Menus interativos de sistemas e validação de entrada digitada pelo teclado.

VALIDACAO_NOTA_DO_WHILE.C

```c
float nota;

// O bloco executa ao menos uma vez; se o usuário digitar -3 ou 15, ele prende e pede de novo!
do {
    printf("Digite uma nota valida entre 0.0 e 10.0: ");
    scanf("%f", &nota);

    if (nota < 0.0 || nota > 10.0) {
        printf("Valor invalido! Tente novamente.\n");
    }
} while (nota < 0.0 || nota > 10.0); // Note o ';' obrigatório aqui!
```

8.2 O Laço `for` (Para): A Repetição Contada por Excelência

Quando sabemos exatamente quantas voltas o laço precisa dar (ex: repetir 10 vezes, gerar a tabuada do 1 ao 10 ou percorrer todas as posições de um vetor), o laço `for` é a ferramenta ideal. Ele concentra no seu cabeçalho, separados por dois pontos-e-vírgulas, os três elementos vitais de qualquer repetição controlada: onde o contador começa, até onde ele vai e de quanto em quanto ele avança.

ANATOMIA E CICLO DE EXECUÇÃO DO LAÇO FOR

Preste muita atenção na ordem numérica: o Passo 4 (incremento) só ocorre DEPOIS do Passo 3 (corpo do laço)!

```
for ( [1] int i = 0; [2] i < 5; [4] i++ ) { [3] printf("%d ", i); }
```

PASSO 1

Inicialização

`int i = 0;`

Executada **apenas uma única vez** quando o programa alcança o `for`, preparando o valor inicial do contador.

PASSO 2

Teste Condicional

`i < 5;`

Avaliado **antes de cada volta**. Se resultar em verdadeiro, entra no Passo 3; se resultar em falso, encerra o laço imediatamente.

PASSO 3

Corpo do Laço

`printf("%d ", i);`

Executa todas as linhas que estão entre as chaves `{ }` do `for` para a volta atual.

PASSO 4

Passo de Incremento

`i++ (i = i + 1)`

Ao bater na chave de fechamento `}`, soma 1 à variável `i` e volta imediatamente para o **Passo 2**.

Figura 8.1 — Ordem cronológica real de funcionamento do laço for: 1 → (2 → 3 → 4) → (2 → 3 → 4)... até o Passo 2 dar Falso.

8.3 Contadores, Acumuladores e o "Teste de Mesa" Passo a Passo

Dentro dos laços de repetição, utilizamos constantemente dois padrões de variáveis que todo programador precisa dominar:

- **Variável Contadora (Ex: `cont++` ou `cont = cont + 1`):** Aumenta de um valor constante (geralmente de 1 em 1) para contar quantas vezes um evento aconteceu (por exemplo, contar quantos alunos tiraram nota acima de 7). Na linguagem C, o operador `i++` é um atalho exato para `i = i + 1`, e `i--` equivale a `i = i - 1`.
- **Variável Acumuladora (Ex: `soma += valor` ou `soma = soma + valor`):** Vai somando valores variáveis que chegam a cada volta do laço, funcionando como o visor de uma caixa registradora de supermercado. **Regra vital:** todo acumulador de soma deve obrigatoriamente nascer inicializado com zero (`soma = 0;`) antes do laço começar!

A melhor maneira de ter certeza absoluta de que o seu laço funciona — e uma técnica cobrada em quase todas as provas de programação — é fazer um **Teste de Mesa (*Dry Run*)**: desenhar uma tabela no papel com uma coluna para cada variável e simular o computador linha por linha. Veja o código abaixo e seu Teste de Mesa completo:

```c
int soma = 0;
for (int i = 1; i <= 4; i++) {
    soma = soma + (i * 10);
}
printf("Soma Final = %d\n", soma);
```

Momento / Volta	Valor de `i`	Teste `i <= 4`?	Cálculo no Corpo: `soma = soma + (i * 10)`	Novo Valor de `soma`
Antes do for	—	—	Inicialização da variável acumuladora	`0`
1ª Volta	`1`	`1 <= 4` → **Verdadeiro**	`soma = 0 + (1 * 10)`	`10`
2ª Volta	`2`	`2 <= 4` → **Verdadeiro**	`soma = 10 + (2 * 10)`	`30`
3ª Volta	`3`	`3 <= 4` → **Verdadeiro**	`soma = 30 + (3 * 10)`	`60`
4ª Volta	`4`	`4 <= 4` → **Verdadeiro**	`soma = 60 + (4 * 10)`	`100`
Fim do Laço	`5` (após `i++`)	`5 <= 4` → **Falso (Sai!)**	Pula fora do laço e imprime `Soma Final = 100`	`100`

DIVIDIR PARA PROGRAMAR

Modularização e Funções: Dividir para Conquistar

Como decompor sistemas complexos em subprogramas independentes, organizados e fáceis de manter.

À medida que os nossos programas crescem de 20 para 300 ou 500 linhas, colocar todo o código empilhado dentro da função `main()` transforma o arquivo no que os engenheiros de software chamam de **código espaguete**: um emaranhado confuso onde é quase impossível encontrar um erro ou reaproveitar um cálculo em outra parte do sistema.

A solução clássica da ciência da computação é aplicar o princípio de **Dividir para Conquistar (Modularização)**: quebrar um problema grande em subproblemas menores e delegar cada tarefa para uma **Função** específica. Na verdade, você já usa funções prontas desde o primeiro dia: `printf` e `scanf` nada mais são do que funções escritas pelos criadores da biblioteca padrão! Agora, aprenderemos a criar as nossas próprias funções.

9.1 Anatomia de uma Função: Retorno, Nome e Parâmetros

Pense em uma função como uma máquina industrial ou um liquidificador: você entrega ingredientes pela abertura superior (chamados de **Parâmetros de Entrada**), a máquina processa esses ingredientes internamente com suas lâminas (o **Corpo da Função**) e, ao final, despeja o suco pronto (o **Valor de Retorno** devolvido pelo comando `return`).

A declaração de qualquer função em C segue sempre o mesmo cabeçalho:

```
tipo_de_retorno nomeDaFuncao(tipo1 parametro1, tipo2 parametro2) {
    // Variáveis locais e cálculos internos da função...
    return resultado; // Devolve o valor calculado para quem chamou a função!
}
```

- **Funções com Retorno (`int`, `float`, `double`, `char`):** Realizam um cálculo e obrigatoriamente encerram com a instrução `return valor;`, devolvendo uma resposta do mesmo tipo anunciado no cabeçalho.
- **Procedimentos Sem Retorno (Tipo `void`):** Em inglês, *void* significa "vazio". Uma função declarada como `void exibirMenu(void)` apenas executa uma tarefa direta (como desenhar um cabeçalho na tela) e não devolve nenhum número para a `main`.

COMO FUNCIONA A PASSAGEM DE PARÂMETROS POR VALOR NA MEMÓRIA RAM

Cada função possui sua própria bancada isolada de variáveis locais na memória (Stack Frame)



Figura 9.1 — Passagem por valor: 'n1' e 'n2' ocupam gavetas próprias na RAM e recebem cópias de 'notaA' e 'notaB'.

9.2 Escopo Local vs. Escopo Global e Exemplo Completo

Observe atentamente a Figura 9.1: as variáveis `notaA` e `notaB` existem exclusivamente dentro da função `main()`, enquanto `n1`, `n2` e `media` existem exclusivamente dentro de `calcularMedia()`. Isso é o que chamamos de **Escopo Local**.

Quando a função `calcularMedia` termina sua execução e atinge o comando `return`, todas as suas variáveis locais (`n1`, `n2`, `media`) são **automaticamente destruídas e liberadas da Memória RAM**, restando apenas o valor numérico devolvido para a `main`. Graças à passagem por valor, se uma função alterar o valor de um parâmetro internamente, a variável original lá na `main` permanece intacta.

MODULARIZACAO_COMPLETA.C

```c
#include <stdio.h>

// 1. Procedimento void: apenas imprime uma linha decorativa na tela
void imprimirLinha(void) {
    printf("-----------------------------------------\n");
}

// 2. Função que recebe dois floats por valor e devolve o maior entre eles
float obterMaiorNota(float a, float b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}

// 3. Função que calcula e devolve a média aritmética
float calcularMedia(float n1, float n2) {
    return (n1 + n2) / 2.0;
}

int main(void) {
    float p1 = 6.5, p2 = 9.0;

    imprimirLinha();
    printf("Maior nota obtida pelo estudante: %.2f\n", obterMaiorNota(p1, p2));
    printf("Media final calculada:            %.2f\n", calcularMedia(p1, p2));
    imprimirLinha();

    return 0;
}
```

MUITOS DADOS, UMA SÓ CAIXA

Vetores (Arrays Unidimensionais) e Manipulação de Strings

Como armazenar coleções sequenciais de dados na memória RAM e dominar o processamento de textos em C.

Considere o seguinte problema: você precisa ler a nota de 5 alunos, calcular a média geral da turma e, logo depois, imprimir quais alunos ficaram acima da média calculada. Se você usar apenas uma variável `nota` dentro de um laço `for`, cada nova nota digitada apagará a nota anterior da memória! Quando o laço terminar, você terá a média, mas terá perdido as 4 primeiras notas.

Para preservar dezenas, centenas ou milhares de valores na memória sem precisar criar variáveis soltas (`n1, n2, n3... n100`), a linguagem C oferece o **Vetor (*Array Unidimensional*)**. Um vetor é uma **estrutura de dados homogênea**: um bloco único e contínuo de gavetas vizinhas na Memória RAM, onde todas as gavetas possuem o **mesmo tipo de dado** e compartilham o **mesmo nome**, sendo diferenciadas apenas pelo número da sua posição, chamado de **Índice**.

ALOCAÇÃO CONTÍGUA DE UM VETOR NA MEMÓRIA RAM

Declaração: int notas[5] = {8, 7, 10, 6, 9}; — Cada int ocupa 4 bytes consecutivos

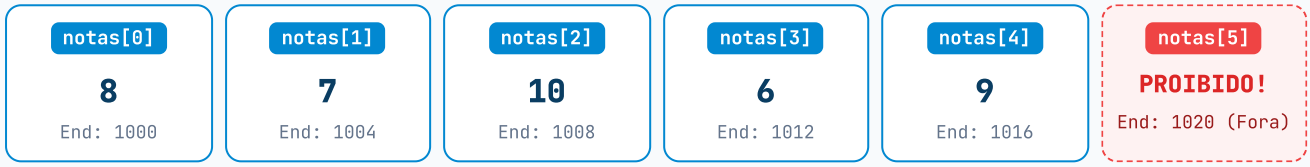


Figura 10.1 — Em um vetor de tamanho N = 5, os índices válidos vão estritamente de 0 até 4 (N - 1). A posição [5] não pertence ao vetor!

10.1 Por que Todo Vetor em C Começa no Índice Zero (`[0]`)?

Quem começa a programar sempre pergunta: *"Por que a primeira posição do vetor é* `notas[0]` *e não* `notas[1]` *?"*. A resposta vem diretamente da arquitetura física da Memória RAM (veja os endereços na base da Figura 10.1):

Para o compilador C, o número dentro dos colchetes `[i]` não é um número ordinal ("primeiro, segundo..."), mas sim a **distância (*offset* / deslocamento) a partir do endereço inicial do vetor**! Como o vetor começa no endereço `1000` e cada `int` tem 4 bytes, o computador encontra qualquer posição instantaneamente fazendo a conta: `Endereço = 1000 + (i × 4)`.

- Para acessar a primeira gaveta, a distância a partir do início é **zero** (`1000 + 0 × 4 = 1000`), logo o índice é `[0]`.
- Para acessar a quinta e última gaveta de um vetor de 5 posições, a distância é 4 gavetas para a frente (`1000 + 4 × 4 = 1016`), logo o último índice válido é sempre `[Tamanho - 1]` (neste caso, `[4]`).

O Crime de Invasão de Memória (*Out-of-Bounds Access*): Quando você declara `int notas[5];`, o número 5 na declaração significa "reserve 5 gavetas (0, 1, 2, 3 e 4)". Se, dentro do código ou em um laço `for (i = 0; i <= 5; i++)`, você tentar gravar um valor em `notas[5]`, o compilador C não te impedirá, mas você estará escrevendo no endereço `1020`, que pertence a outra variável do programa, corrompendo dados ou travando tudo! Use sempre `i < 5` (estritamente menor que o tamanho) no laço `for`.

ANALISE_NOTAS_VETOR.C

```c
#include <stdio.h>

int main(void) {
    float notas[5], soma = 0.0, media, maior;
    int i;

    // 1° Laço: Leitura das 5 notas (índices 0, 1, 2, 3 e 4)
    for (i = 0; i < 5; i++) {
        printf("Digite a nota do aluno [%d]: ", i);
        scanf("%f", &notas[i]);
        soma += notas[i];
    }

    media = soma / 5.0;
    maior = notas[0]; // Assume inicialmente que a primeira nota é a maior

    // 2° Laço: Consulta os valores preservados no vetor!
    printf("\nMedia da Turma = %.2f\nAlunos acima da media:\n", media);
    for (i = 0; i < 5; i++) {
        if (notas[i] > media) {
            printf(" - Aluno [%d] com nota %.2f\n", i, notas[i]);
        }
        if (notas[i] > maior) {
            maior = notas[i]; // Atualiza o recorde se encontrar nota superior
        }
    }
    printf("Maior nota da sala: %.2f\n", maior);
    return 0;
}
```

10.2 Strings em C: Vetores de Caracteres e a Biblioteca <string.h>

Diferentemente de outras linguagens, o C não possui um tipo primitivo chamado `string`. Na linguagem C, qualquer palavra ou frase é armazenada dentro de um **vetor de caracteres (`char`)**, onde cada letra ocupa 1 gaveta de 1 byte.

Mas surge uma questão inteligente: se você declarar um vetor com espaço para 20 letras (`char curso[20];`) e guardar nele a sigla `"UTFPR"` (que tem apenas 5 letras), como o `printf("%s", curso)` sabe que deve parar de imprimir na letra `R` e não continuar imprimindo o lixo de memória das 15 gavetas restantes?

A resposta é o **Caractere Terminador Nulo (`'\0'`)**! Sempre que guardamos um texto entre aspas duplas em C, o compilador grava automaticamente o byte especial `'\0'` logo na gaveta seguinte à última letra: `['U']['T']['F']['P']['R']['\0']`. Assim, para armazenar um texto de até 49 letras úteis, devemos declarar `char nome[50];` (reservando 1 posição extra para o `'\0'`).

Operação com Texto	Forma Errada em C	Forma Correta com `#include <string.h>`
Ler texto no `scanf`	`scanf("%s", &nome);` (Em vetores de char não se usa &!)	`scanf("%s", nome);` (palavra única) `scanf(" %[^\n]", nome);` (lê frase com espaços!)
Atribuir / Copiar texto	`nome = "Renan";` (Proibido atribuir vetor com = após criado)	`strcpy(nome, "Renan");` (String Copy: copia letra por letra até o `'\0'`)
Comparar dois textos	`if (senha == "1234")` (Compara endereços e dá sempre Falso!)	`if (strcmp(senha, "1234") == 0)` (String Compare: retorna `0` se forem idênticas!)
Medir tamanho do texto	Contar gavetas vazias do vetor	`int tam = strlen(nome);` (String Length: conta quantas letras há antes do `'\0'`)

TABELAS, GRADES E COORDENADAS

Matrizes (Arrays Bidimensionais): Tabelas, Grades e Jogos

Organizando dados em linhas e colunas e percorrendo grades bidimensionais com laços for aninhados.

Enquanto o vetor organiza os dados em uma única fila horizontal, muitos problemas do mundo real são naturalmente bidimensionais: uma planilha de notas onde cada linha é um aluno e cada coluna é uma matéria; um tabuleiro de Xadrez ou Jogo da Velha; ou uma imagem digital na tela do seu celular, que nada mais é do que uma grade de milhões de pixels dispostos em linhas e colunas.

Para representar tabelas, a linguagem C oferece a **Matriz (*Array Bidimensional*)**. Na declaração `float boletim[4][3];`, o primeiro par de colchetes define sempre a quantidade de **Linhas** (horizontal) e o segundo par define a quantidade de **Colunas** (vertical). Neste exemplo, temos uma tabela de 4 × 3 = 12 células na memória!

MAPA DE COORDENADAS DE UMA MATRIZ QUADRADA 3X3: INT MAPA[3][3];

Para acessar qualquer célula, informamos primeiro a Linha [i] e depois a Coluna [j]

	Coluna j = 0	Coluna j = 1	Coluna j = 2
Linha i = 0	mapa[0][0] Diag. Principal	mapa[0][1]	mapa[0][2] Diag. Secundária
Linha i = 1	mapa[1][0]	mapa[1][1] Centro (Ambas)	mapa[1][2]
Linha i = 2	mapa[2][0] Diag. Secundária	mapa[2][1]	mapa[2][2] Diag. Principal

Propriedades Geométricas das Matrizes:

- **Sincronismo dos Ponteiros do Relógio:** O laço externo (`i`) funciona como o ponteiro das horas (fixa a linha atual), enquanto o laço interno (`j`) funciona como o ponteiro dos minutos (percorre todas as colunas daquela linha da esquerda para a direita).
- **Diagonal Principal (Azul):** Células onde o número da linha é igual ao da coluna: `i == j`.
- **Diagonal Secundária (Âmbar):** Em uma matriz N × N, ocorre onde `i + j == N - 1`.

Figura 11.1 — Coordenadas [i][j] de uma matriz 3×3, destacando a Diagonal Principal (i == j) e a Diagonal Secundária (i + j == 2).

11.1 Varredura Completa, Impressão Tabular e Soma por Linha

Assim como um vetor de 1 dimensão precisa de 1 laço `for`, uma matriz de 2 dimensões exige **2 laços `for` aninhados (um dentro do outro)**. Veja no exemplo completo abaixo como ler uma matriz 3 × 3, imprimi-la no formato visual de tabela e calcular simultaneamente a soma de cada linha e o traço da diagonal principal:

OPERACOES_MATRIZ_3X3.C

```c
#include <stdio.h>

int main(void) {
    int mat[3][3];
    int i, j, soma_diagonal = 0;

    // 1. Leitura dos 9 elementos da matriz usando dois laços for aninhados:
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("Digite o valor da coordenada [%d][%d]: ", i, j);
            scanf("%d", &mat[i][j]);
        }
    }

    // 2. Impressão no formato de tabela + Cálculo da soma de cada linha:
    printf("\n--- TABELA DA MATRIZ 3x3 ---\n");
    for (i = 0; i < 3; i++) {
        int soma_linha = 0; // Zera o acumulador ao começar cada nova linha i!

        for (j = 0; j < 3; j++) {
            printf("%4d ", mat[i][j]); // %4d alinha os números em colunas de 4 espaços
            soma_linha += mat[i][j];

            if (i == j) {
                soma_diagonal += mat[i][j]; // Captura os elementos da Diagonal Principal
            }
        }
        // Ao terminar todas as colunas j da linha atual, imprime a soma e pula de linha (
)!
        printf(" | Soma da Linha %d = %d\n", i, soma_linha);
    }

    printf("\nSoma da Diagonal Principal: %d\n", soma_diagonal);
    return 0;
}
```

MODELANDO ENTIDADES DO MUNDO REAL

Structs: Criando Nossos Próprios Tipos de Dados

Como agrupar informações de diferentes tipos dentro de um único registro estruturado e criar vetores de fichas.

Ao estudarmos vetores e matrizes na Fase 5, vimos que eles são estruturas *homogêneas*: um vetor de `int` só aceita números inteiros; um vetor de `float` só aceita números reais. Mas pare para pensar em como são os cadastros do mundo real: a ficha de um **Estudante da UTFPR** contém simultaneamente um Nome (`char[50]`), um Registro Acadêmico RA (`int`), uma Média Geral (`float`) e uma Situação de Matrícula (`char`). Um **Produto de E-commerce** possui Descrição, Código de Barras, Quantidade em Estoque e Preço Unitário.

Para que não precisemos criar vários vetores separados e correr o risco de misturar os dados de pessoas diferentes, a linguagem C oferece a `struct` **(Estrutura Heterogênea ou Registro)**. Com uma `struct` , você atua como arquiteto da linguagem e cria um **novo tipo de dado personalizado** capaz de guardar várias variáveis internas (chamadas de *campos* ou *membros*) dentro de uma única gaveta composta.

ANATOMIA DE UMA STRUCT: O MOLDE ABSTRATO VS. A FICHA ALOCADA NA MEMÓRIA RAM

Para acessar cada compartimento interno da ficha, utilizamos o Operador Ponto (variavel.campo)

```c
// 1. Criação do Molde (Fora da main):
struct Aluno {
  char nome[50];
  int ra;
  float media;
  char ativo;
}; // ← Nunca esqueça este ';'

// 2. Declaração da Variável:
struct Aluno a1;
```

BLOCO NA MEMÓRIA RAM: Variável "a1" tipo: struct Aluno

a1.nome	a1.ra
char[50] (String)	int (4 bytes)
"Mariana Silva"	240189
a1.media	**a1.ativo**
float (4 bytes)	char (1 byte)
9.50	'S'

Figura 12.1 — O bloco 'struct Aluno' funciona como uma planta de arquitetura: ele só passa a ocupar memória RAM quando criamos a variável 'a1'.

12.1 O Operador Ponto (.) e o Comando typedef

Uma vez declarada a variável `struct Aluno a1;` , não podemos fazer um `scanf` ou `printf` da ficha inteira de uma só vez. Em vez disso, acessamos cada compartimento interno colocando um **ponto (`.`)** entre o nome da variável e o nome do campo desejado: `a1.ra = 240189;` ou `scanf("%f", &a1.media);` .

Para evitar ter que escrever a palavra `struct` toda vez que declararmos uma variável, os programadores de C costumam combinar a estrutura com o comando `typedef` (definidor de apelido de tipo), permitindo declarar simplesmente `Aluno a1;` .

12.2 O Ápice do Curso: Vetores de Structs (Sistema de Cadastro)

Quando unimos o poder dos **Vetores (Capítulo 10)** com o poder das **Structs (Capítulo 12)** e dos **Laços `for` (Capítulo 8)**, alcançamos a capacidade de construir sistemas reais de banco de dados em memória. Ao declarar `struct Aluno turma[3];` , criamos um fichário com 3 fichas completas (`turma[0]` , `turma[1]` e `turma[2]`), onde cada posição `[i]` do vetor guarda um aluno inteiro com nome, RA e média:

SISTEMA_ACADEMICO_STRUCTS.C

```c
#include <stdio.h>

// Definição do molde com typedef: cria o novo tipo 'Aluno'
typedef struct {
    char  nome[50];
    int   ra;
    float media;
} Aluno;

int main(void) {
    Aluno turma[3]; // Vetor de Structs: 3 fichas completas na memória RAM!
    int i;

    for (i = 0; i < 3; i++) {
        printf("\n--- Cadastro do Estudante #%d ---\n", i + 1);
        printf("Nome completo: ");
        scanf(" %[^\n]", turma[i].nome); // Lê string com espaços (sem &)

        printf("RA e Media Final: ");
        scanf("%d %f", &turma[i].ra, &turma[i].media); // Campos numéricos levam &
    }

    printf("\n================ RELATORIO OFICIAL DA TURMA ================\n");
    for (i = 0; i < 3; i++) {
        printf("RA: %6d | Nome: %-20s | Media: %.2f\n",
               turma[i].ra, turma[i].nome, turma[i].media);
    }
    return 0;
}
```

O MÉTODO TRY, CATCH & LEARN

Pensando como um Programador: Guia de Depuração

O roteiro de 4 passos para resolver qualquer exercício de prova e o dicionário tradutor de erros do compilador GCC.

Ao longo de nossa experiência acompanhando colegas nas disciplinas de programação, notamos que o maior erro diante de um exercício difícil é abrir o OnlineGDB e começar a digitar código imediatamente por impulso. Programar não é digitar rápido: **80% do trabalho acontece no papel e no raciocínio lógico, e apenas 20% consiste em traduzir essa ideia para a sintaxe da linguagem C**.

O Protocolo de 4 Passos antes de Digitar a Primeira Linha

1. **Separe Entrada, Processamento e Saída no Papel:** Leia o enunciado grifando: *O que o usuário vai digitar no teclado (`scanf`)? Qual fórmula ou regra transforma esses dados? O que precisa ser impresso no final (`printf`)?*
2. **Simule um Caso Real à Mão:** Antes de escrever uma única linha em C, invente valores simples (ex: nota 6 e nota 8) e faça a conta no papel. Se você não souber como resolver o problema no papel, não conseguirá ensiná-lo ao computador.
3. **Construa e Teste em Pequenos Blocos:** Se o exercício pede para ler uma matriz, somar as linhas e achar o maior valor, não escreva tudo de uma vez! Escreva apenas a leitura, clique em *Run* e teste. Funcionou? Só então adicione o cálculo seguinte.
4. **Teste os Casos de Fronteira (*Edge Cases*):** Seu programa funcionou para o número 10? Agora teste digitar `0`, números negativos ou exatamente o valor limite da condição (como média `7.0`) para garantir que você não trocou `>=` por `>`.

Dicionário Tradutor das Mensagens Reais do Compilador GCC

Mensagem em Inglês no Console do GCC	O que Aconteceu de Verdade e Como Consertar
`error: expected ';' before '...'`	Faltou colocar o ponto-e-vírgula (`;`) no final da **linha imediatamente anterior** à linha indicada pelo número do erro.
`error: 'x' undeclared (first use in this function)`	Você tentou usar uma variável chamada `x` sem antes declará-la com seu tipo (ex: `int x;`), ou digitou o nome dela com uma letra maiúscula/minúscula diferente da declaração.
`warning: format '%d' expects argument of type 'int *', but argument 2 has type 'int'`	Alerta clássico do `scanf`: você escreveu `scanf("%d", idade);` e **esqueceu de colocar o símbolo de endereço** `&` antes do nome da variável (`&idade`)!
`undefined reference to 'main'` ou `to 'prinft'`	Erro do Linker: você escreveu o nome de uma função com erro de grafia (`prinft` em vez de `printf`) ou esqueceu de criar a função `int main()`.
`Segmentation fault (core dumped)`	O programa tentou acessar uma área proibida da Memória RAM. Causas quase certas: faltou `&` no `scanf`, ou o seu laço `for` ultrapassou o tamanho máximo do vetor/matriz!

Mensagem Final da Equipe Try, Catch & Learn: Ninguém nasce sabendo programar. Até mesmo os engenheiros mais experientes do mundo consultam a documentação e corrigem erros de compilação todos os dias. Cada bug que você investiga com calma hoje constrói a intuição lógica que fará de você um excelente profissional amanhã. Bons estudos e excelente jornada na computação!

FUNDAMENTAÇÃO CIENTÍFICA E TÉCNICA (ABNT NBR 6023)

Referências Bibliográficas e Documentais

Relação completa das obras clássicas, artigos históricos, normas técnicas e manuais reais utilizados na elaboração deste livro.

Todo o conteúdo histórico, teórico e prático apresentado neste ebook foi elaborado e verificado com base na literatura canônica de Ciência da Computação, nos artigos originais dos criadores da arquitetura moderna e nas especificações oficiais da linguagem C e do compilador GCC. Abaixo estão listadas as fontes reais organizadas por categoria segundo a norma **ABNT NBR 6023**:

1. Livros-Texto Clássicos de Linguagem C e Algoritmos

KERNIGHAN, Brian Wilson; RITCHIE, Dennis MacAlistair. *C: a linguagem de programação padrão ANSI*. Tradução de Daniel Vieira. Rio de Janeiro: Campus / Elsevier, 1989. 290 p. ISBN 978-8570015860. (Obra original: *The C Programming Language*. 2nd ed. Englewood Cliffs: Prentice Hall, 1988. ISBN 978-0131103627).

Referência primária mundial da linguagem C, escrita pelo próprio criador da linguagem (Dennis Ritchie) e por Brian Kernighan nos Laboratórios Bell. Base para os Capítulos 1, 4, 5, 10 e 12.

DEITEL, Paul; DEITEL, Harvey. *C: como programar*. 6. ed. Tradução de Edson Furmankiewicz; revisão técnica de Fábio Lucchini. São Paulo: Pearson Prentice Hall, 2011. 818 p. ISBN 978-8576059349.

Bibliografia básica adotada nos cursos de Computação e Engenharias da UTFPR. Base didática para estruturas de controle (Capítulos 7 e 8), modularização em funções (Capítulo 9) e boas práticas de programação segura.

BACKES, André Ricardo. *Linguagem C: completa e descomplicada*. 2. ed. Rio de Janeiro: Elsevier / GEN LTC, 2019. 408 p. ISBN 978-8535291063.

Obra de referência nacional (UFU) amplamente utilizada no ensino superior brasileiro pela abordagem visual de alocação na memória RAM, limpeza de buffer de teclado, vetores, matrizes e structs (Capítulos 5, 6, 10, 11 e 12).

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. *Lógica de programação: a construção de algoritmos e estruturas de dados*. 3. ed. São Paulo: Prentice Hall, 2005. 218 p. ISBN 978-8576050247.

Clássico paranaense de construção algorítmica. Fundamentou a metodologia de decomposição lógica, fluxogramas de decisão, contadores, acumuladores e tabelas de Teste de Mesa (Capítulos 7, 8 e 13).

MIZRAHI, Victorine Viviane. *Treinamento em linguagem C*. 2. ed. São Paulo: Pearson Prentice Hall, 2008. 432 p. ISBN 978-8576051916.

Referência clássica para especificadores de formato de entrada e saída (`printf`/`scanf`), precedência de operadores aritméticos e manipulação de cadeias de caracteres com `<string.h>` (Capítulos 4, 5, 6 e 10).

SCHILDT, Herbert. *C completo e total*. 3. ed. Tradução de Roberto Carlos Mayer. São Paulo: Makron Books / Pearson Education, 1997. 827 p. ISBN 978-8534605953.

Manual abrangente sobre a biblioteca padrão ANSI C, tipos de dados primitivos, escopo de variáveis locais/globais e estruturas heterogêneas (Capítulos 5, 9 e 12).

ASCENCIO, Ana Fernanda Gomes; CAMPOS, Edilene Aparecida Veneruchi de. *Fundamentos da programação de computadores: algoritmos, PASCAL, C/C++ (padrão ANSI) e JAVA*. 3. ed. São Paulo: Pearson Education do Brasil, 2012. 569 p. ISBN 978-8564574168.

Utilizado na estruturação comparativa de estruturas de repetição (`while`, `do-while`, `for`), vetores unidimensionais e matrizes bidimensionais (Capítulos 8, 10 e 11).

2. Artigos Históricos Seminais da Computação

RITCHIE, Dennis MacAlistair. The development of the C language. *ACM SIGPLAN Notices*, New York, v. 28, n. 3, p. 201–208, mar. 1993. DOI: 10.1145/155360.155580.

Artigo histórico escrito por Dennis Ritchie detalhando a evolução das linguagens BCPL e B até a criação da linguagem C e a reescrita do kernel do sistema UNIX nos Laboratórios Bell entre 1969 e 1973 (Capítulo 1).

TURING, Alan Mathison. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, London, s2–42, n. 1, p. 230–265, nov. 1936. DOI: 10.1112/plms/s2-42.1.230.

Artigo fundador da Ciência da Computação moderna que formalizou o conceito matemático de algoritmo e de Máquina Universal de programa armazenado (Capítulo 1).

MENABREA, Luigi Federico; LOVELACE, Ada Augusta King (Trad. e Notas). Sketch of the Analytical Engine invented by Charles Babbage, Esq. With notes upon the Memoir by the Translator. *Taylor's Scientific Memoirs*, London, v. 3, Article XXIX, p. 666–731, 1843.

Publicação histórica contendo a "Nota G" de Ada Lovelace com o primeiro algoritmo documentado da história (cálculo dos Números de Bernoulli) e a distinção entre processamento numérico e simbólico (Capítulo 1).

3. Normas Internacionais, Manuais Técnicos e Diretrizes REA

ISO/IEC. *ISO/IEC 9899:2018 — Information technology — Programming languages — C (C17 Standard)*. Geneva: International Organization for Standardization, 2018.

Norma técnica internacional oficial que regulamenta a sintaxe, os tipos de dados e a biblioteca padrão da linguagem C.

FREE SOFTWARE FOUNDATION. *Using the GNU Compiler Collection (GCC)*. Boston: Free Software Foundation, 2023. Disponível em: https://gcc.gnu.org/onlinedocs/.

Documentação técnica oficial do compilador GCC utilizada para descrever as quatro etapas do pipeline de compilação (`cpp`, `cc1`, `as`, `ld`) e as mensagens de diagnóstico de erro (Capítulos 2 e 13).

UNESCO. *Declaração REA de Paris em 2012*. Congresso Mundial sobre Recursos Educacionais Abertos (REA). Paris: Organização das Nações Unidas para a Educação, a Ciência e a Cultura, 2012. Disponível em: https://unesdoc.unesco.org/ark:/48223/pf0000246687_por.

Documento norteador dos princípios de democratização, acessibilidade universal e licenciamento aberto adotados pelo projeto Try, Catch & Learn.

MARÇULO, Lauren; EDUARDA, Lorena; VIEIRA, Manuella; ANSELMO, Renan Cáceres; DOMINGUES, André Luís dos Santos (Orient.). *Projeto Try, Catch & Learn: Recursos Educacionais Abertos para Fundamentos de Programação em C (Fases 1 a 6 e Cadernos de Missões)*. Cornélio Procópio: Universidade Tecnológica Federal do Paraná (UTFPR-CP), 2026. Licenciado sob MIT License.

Acervo didático original desenvolvido pela equipe autora na disciplina Certificadora de Competência Comum da UTFPR — Câmpus Cornélio Procópio, do qual este livro sintetiza e expande toda a trilha formativa.